Creare video e contenuti multimediali

senza strumenti a pagamento

Guida pratica, passo per passo

Undici strumenti gratuiti, una tecnica e i progetti: come far lavorare un agente sui propri profili, come montare tutto con ffmpeg, e l'indice dei cantieri.

Con esempi, comandi e gli avvisi su ciò che è gratuito e ciò che non lo è.

Documento di lavoro — verificato sulle fonti ufficiali

Perché questa guida

Il multimedia si può fare senza abbonamenti. Non è una promessa da volantino: è che negli ultimi anni gli strumenti **aperti** hanno raggiunto quelli commerciali, e in alcuni casi li hanno superati — perché chi costruisce un prodotto chiuso deve proteggere il proprio margine, mentre chi costruisce uno strumento aperto può permettersi di essere usato da chiunque, ovunque, senza chiedere il conto.

Questa guida mette in fila undici strumenti e **una tecnica**. Gli strumenti si scelgono in base a cosa devi fare; la tecnica, invece, cambia il modo in cui lavori, ed è per questo che ha un capitolo a sé.

> *Il criterio con cui ho scritto: per ogni strumento dico **cosa fa**, **quanto costa davvero** e **cosa serve per usarlo**. Dove un prodotto è a pagamento lo dico apertamente: non serve a nessuno un elenco che promette gratuità e poi chiede la carta di credito al primo salvataggio.*

Il quadro d'insieme: la catena del lavoro

Prima degli strumenti, conviene vedere la filiera. Un video, un tutorial, una presentazione multimediale nascono sempre da cinque passaggi:

1. **Il materiale** — quello che si vede: una registrazione dello schermo, una pagina web, un'immagine, un oggetto 3D.
2. **Il montaggio** — tagliare, unire, dissolvenze, formato.
3. **La voce** — il parlato che accompagna.
4. **Il ritmo e l'animazione** — movimenti, comparse, sottotitoli.
5. **La pubblicazione** — il formato giusto per il posto giusto.

Ogni strumento di questa guida lavora su uno o più di questi passaggi. Alla base di tutti, però, c'è **una cosa sola**: `ffmpeg`. È il tornio su cui girano anche i programmi con l'interfaccia grafica. Capirlo è il modo più veloce per smettere di dipendere da un'applicazione specifica.

1 · FFmpeg: la base di tutto

`ffmpeg` è un programma a riga di comando che legge, trasforma e scrive audio e video. Non ha finestre, non ha pulsanti: gli si dice cosa fare e lo fa. È **libero e gratuito** (licenza LGPL 2.1 o superiore; alcune parti opzionali come il codificatore `libx264` sono GPL 2, e chi le include porta l'insieme sotto GPL), gira su Windows, Mac e Linux, e — questa è la parte che conta — **quasi tutti gli strumenti con l'interfaccia grafica lo usano dentro**: quando un programma "esporta in MP4", nella maggior parte dei casi sta chiamando ffmpeg.

> *Un dettaglio che rassicura: per ffmpeg **non esiste una versione a pagamento**, nemmeno volendo. Non c'è un piano "pro": c'è il programma, e la licenza libera. È il motivo per cui vale la pena investirci tempo.*

Imparare cinque comandi di ffmpeg vale più che imparare cinque programmi.

Cosa sa fare, in concreto

- **Convertire** un formato in un altro (`.mov` → `.mp4`, `.webm` → `.mp4`).
- **Ritagliare** un pezzo di video (dal secondo 12 al secondo 30).
- **Unire** più video in uno solo.
- **Cambiare formato e risoluzione** (portare tutto a 1920×1080, il formato 16:9).
- **Estrarre l'audio** da un video, o **sostituirlo**.
- **Aggiungere una traccia musicale** e abbassarla automaticamente quando qualcuno parla.
- **Dissolvenze** fra una scena e l'altra.
- **Registrare lo schermo** e la **voce del computer**.

I cinque comandi che servono davvero

Convertire e portare tutto al formato di pubblicazione (16:9, 1920×1080):

```
ffmpeg -i sorgente.mov -vf "scale=1920:1080" -c:v libx264 -crf 20 -pix_fmt yuv420p uscita.mp4
```

Ritagliare un pezzo (dal minuto 0:12 per 18 secondi):

```
ffmpeg -ss 12 -t 18 -i video.mp4 -c copy pezzo.mp4
```

Unire due video (con un file di elenco `lista.txt` che contiene le righe `file 'uno.mp4'`):

```
ffmpeg -f concat -safe 0 -i lista.txt -c copy unito.mp4
```

Estrarre l'audio:

```
ffmpeg -i video.mp4 -vn -c:a copy audio.m4a
```

Registrare lo schermo su Windows, **con il cursore del mouse**:

```
ffmpeg -f gdigrab -framerate 15 -draw_mouse 1 -i desktop schermo.mp4
```

Le tre trappole

1. `-crf` **è la qualità.** Più basso è il numero, migliore è l'immagine e più grande il file. `18` è quasi senza perdita, `23` è il compromesso abituale, `28` è già compresso.
2. **La larghezza e l'altezza devono essere pari.** Un video largo 1921 pixel fa fallire la codifica con un errore poco chiaro: si arrotonda a 1920.
3. **Il cursore del mouse non c'è, a meno che non lo si chieda.** Nella registrazione dello schermo, senza `-draw_mouse 1` il video esce senza puntatore, e non c'è modo di aggiungerlo dopo.

2 · Recordly: registrare lo schermo

Un registratore di schermo serve a una cosa sola, ma indispensabile: **mostrare quello che fai**. È lo strumento con cui nascono i tutorial.

Recordly (sito **recordly.dev**) è un registratore di schermo con un editor incorporato: ingrandimento automatico sulle zone che contano, movimento morbido del cursore, bolla della webcam, linea del tempo, esportazione in MP4 o GIF. È **gratuito e open source**, con licenza **AGPL-3.0**: si può usare, studiare e modificare; chi ne distribuisce una versione modificata deve renderne pubblico il sorgente. Gira **in locale**, su macOS 14 o superiore, Windows 10 (build 19041 o superiore) e Linux.

> ***Attenzione a un tranello.*** *Cercando "Recordly" si incontra anche* ***recordly.vip****, che vende un prodotto* ***diverso e a pagamento*** *(circa 20 dollari una volta sola) e* ***non è open source****. Sono due cose che portano lo stesso nome: quello libero è* ***recordly.dev****.*

La logica d'uso è sempre la stessa, in qualunque registratore:

1. si sceglie **cosa** registrare (tutto lo schermo, una finestra, una zona);
2. si sceglie se registrare anche **l'audio** (microfono, suono del sistema, o entrambi);
3. si preme registra, si fa la cosa da mostrare, si preme stop;
4. si esporta in **MP4**.

> *La registrazione va fatta* ***muta e senza fretta****. La voce si aggiunge dopo, e il montaggio la sincronizza: è molto più semplice che parlare mentre si clicca.*

Le regole che fanno la differenza

- **Chiudi tutto il resto.** Una notifica che compare a metà registrazione rovina il video, e se contiene un nome o un messaggio privato è anche un problema di riservatezza. Non è un consiglio di stile: è la differenza fra un video pubblicabile e uno da rifare.
- **Registra in una risoluzione che conosci.** Se lo schermo è più piccolo del formato di destinazione (per esempio 1536×864 quando il video deve essere 1920×1080), il video andrà ingrandito e perderà nitidezza. Meglio saperlo prima.
- **Muovi il mouse con calma.** Il puntatore che sfreccia è illeggibile. Le pause di un secondo, prima e dopo ogni clic, sono quello che rende un tutorial seguibile.
- **Un video muto si può sempre montare.** Un video senza la scena che serviva, no: si rifà.

3 · Playwright: far muovere il browser da solo

Playwright è uno strumento di Microsoft, **libero e gratuito**, che comanda un browser vero — Chrome, Edge, Firefox — dall'interno di un programma. Non serve a registrare: serve a **fare le cose al posto tuo**, in modo ripetibile.

È la differenza fra registrare a mano una procedura e **descriverla una volta** perché poi si esegua da sola.

Cosa permette

- Aprire una pagina e **leggere la sua struttura**: quali elementi si possono cliccare, che testo hanno, dove sono.
- **Muovere il mouse e cliccare** su un elemento preciso, cercandolo per testo o per indirizzo.

- **Scorrere**, compilare, attendere che una pagina sia pronta.
- **Registrare** quello che succede, oppure lasciare che a registrare sia ffmpeg (vedi oltre).
- **Eseguire la stessa procedura quante volte serve**, identica.

Il punto che cambia tutto: la registrazione dallo schermo

Playwright sa produrre da sé un video della pagina, ma è uno *screencast*: **il puntatore del mouse non c'è**, perché il video viene composto dalla pagina e non dallo schermo. Il cursore va quindi disegnato dentro la pagina, con un po' di codice.

C'è però una strada migliore, e sta nel combinare due strumenti:

> ***Playwright pilota, ffmpeg registra.*** *Si apre il browser **visibile**, Playwright muove il mouse vero, e ffmpeg riprende la finestra con* `-draw_mouse 1`*. Risultato: si vedono **il puntatore vero** e **la barra degli indirizzi**, e resta tutto deterministico — perché a decidere le azioni è uno script, non una mano.*

Le due accortezze, entrambe imparate sbagliando:

- **Si registra quello che si vede.** Se la finestra del browser non è davanti a tutte le altre, nel video finisce un'altra applicazione. Prima di registrare va portata in primo piano e **verificato** che ci sia davvero.
- **Le coordinate sono in pixel fisici.** Con lo schermo al 125% di ingrandimento (1920×1080 fisici che diventano 1536×864 logici), un programma che legge le coordinate "logiche" sbaglia la zona da riprendere. Va detto al programma di leggere in pixel fisici.

Un avvertimento onesto

Playwright si presenta ai siti come **browser automatizzato**, e alcuni servizi — Google in testa — rispondono *«questo browser potrebbe non essere sicuro»* **prima ancora di chiedere le credenziali**. Non è un blocco sulla password: non c'è nessun campo dove digitarla.

La soluzione esiste, ed è il capitolo 12. Qui basti sapere che il problema non è la password.

4 · HyperFrames: montare il video da una pagina

Qui si chiude il cerchio fra i tre capitoli precedenti. **Playwright** fa muovere il browser e **ffmpeg** registra quello che si vede: ma un video non è solo una registrazione. È una sequenza: un titolo che compare, una scheda che spiega, la registrazione che scorre, la voce che accompagna, la dissolvenza, la chiusura.

HyperFrames è il motore che fa questo. Si scrive una **composizione** — un file HTML con dentro gli elementi e la loro linea del tempo — e il motore la trasforma in un video. Non è un programma di montaggio con i pulsanti: è **codice**, e questo è precisamente il suo vantaggio.

È **libero e gratuito** (licenza Apache 2.0) e **gira in locale**: il render avviene sul proprio computer, senza servizi da pagare. Richiede due cose che di solito ci sono già: **Node.js** e **ffmpeg**.

L'idea: il video è un progetto, non un file

Un video montato a mano è un file: se domani cambia una frase, si riapre il programma e si rifà la modifica a mano. Una composizione no.

> *Se il video è **codice**, allora il video si **rigenera**. Cambi il testo, rilanci, e il film esce aggiornato — identico a prima in tutto il resto, senza toccare niente con le mani.*

Questa è la proprietà che conta quando i video sono trenta e non uno: non si "aggiustano", si **ricostruiscono**. Ed è la stessa idea di tutto il resto di questa guida — descrivere una procedura una volta, invece di eseguirla ogni volta.

Come si lavora

Cinque passi, in quest'ordine:

1. **Si scrive la composizione.** Gli elementi (testi, immagini, la registrazione, la voce, la musica) e la linea del tempo: cosa compare quando, per quanto, con quale movimento.
2. **Si controlla** (`check`): il motore verifica errori, impaginazione e **contrasto** dei colori — cioè se il testo si legge sopra lo sfondo. Un errore di contrasto viene segnalato con il colore suggerito.
3. **Si guarda.** Non si guarda il video: si chiedono i **fotogrammi** in momenti precisi (`snapshot`). È il modo giusto di lavorare, ed è anche il più veloce: guardare un filmato intero per scoprire che il titolo esce dal bordo è uno spreco.
4. **Si vede in movimento** (`preview`): si apre l'anteprima nel browser, per controllare il ritmo.
5. **Si produce il film** (`render`): il video finale, in locale.

Le trappole, che sono costate tempo

Queste non sono avvertenze di manuale: sono cose che ho scoperto sbagliando, e le scrivo perché chi comincia ci sbatte.

- **Un elemento audio senza identificativo è muto, e non dà nessun errore.** Il video esce, si apre, funziona — e non si sente niente. Va sempre verificato che la voce *ci sia*, ascoltandola, non che il file esista.
- **Il render è lento, e i tempi non si estrapolano.** Su una macchina senza scheda grafica un test di dieci secondi può richiedere un minuto e mezzo, e uno da cento secondi quasi cinque minuti. Da un test breve **non si può dedurre** quanto impiegherà un video lungo: va misurato.
- **Le dipendenze si risolvono solo da una cartella chiamata** `node_modules`**.** Spostare il motore in una cartella con un altro nome lo rompe, e l'errore non lo dice.
- **Il formato di pubblicazione va deciso all'inizio.** Un video 1920×1080 per una piattaforma didattica e un video verticale per i social sono due composizioni diverse: non si converte l'una nell'altra senza perdere qualcosa.

Dove si colloca, in una frase

ffmpeg è il motore che sa fare tutto ma non decide niente; **Playwright** decide le azioni; **HyperFrames** decide il **racconto** — cosa si vede, quando e per quanto. Insieme coprono l'intera filiera, e nessuno dei tre costa qualcosa.

5 · rrweb: registrare il web, non lo schermo

Fin qui abbiamo registrato **pixel**: ffmpeg riprende quello che si vede sullo schermo, e il risultato è un video. rrweb fa una cosa diversa e sorprendente: **registra la pagina, non l'immagine della pagina**.

Si chiama «record and replay the web», è **libero e gratuito** (licenza MIT) ed è lo strumento su cui è costruita la maggior parte dei sistemi di *session replay* — quelli che mostrano come un visitatore ha usato un sito. Invece di catturare fotogrammi, prende nota di **tutti i cambiamenti della pagina**: il testo che compare, un elemento che si sposta, un menu che si apre, un clic. Poi li **riproduce**, ricostruendo la pagina esattamente come si è evoluta.

Perché è una cosa diversa

La differenza non è tecnica: è di sostanza, e si vede in quattro conseguenze.

	registrazione dello schermo (ffmpeg)	rrweb
cosa cattura	i **pixel**	i **cambiamenti della pagina**
dimensione del file	grande (decine di MB)	**piccola** (spesso centinaia di volte meno)
il testo dentro	è un'immagine: **non si può cercare**	è testo vero: **si può cercare e copiare**
risoluzione	quella dello schermo	**indipendente**: si riproduce a qualsiasi dimensione, sempre nitido
cosa non cattura	—	**canvas, video incorporati, animazioni particolari** possono non tornare identici

Il terzo punto è quello che sorprende chi non l'ha mai visto: un video registrato non si può cercare. Un rrweb sì — perché dentro non c'è un'immagine, c'è **la pagina**.

Dove serve davvero

- **Documentare un'interfaccia** in modo che resti **interrogabile**: si registra un percorso e poi lo si ripercorre cercando una parola.
- **Capire cosa è andato storto** per un utente: si riproduce esattamente la sequenza che ha portato al problema, non un racconto.
- **Assistere qualcuno a distanza** mostrandogli il percorso, senza la registrazione pesante.
- **Tutorial leggeri**, quando il file deve restare piccolo e la pagina non ha contenuti complicati.

Perché, in questo progetto, l'ho valutato e non l'ho usato

Questo è il punto onesto, e vale più di una descrizione entusiasta. Quando ho dovuto scegliere come registrare i percorsi di navigazione, ho studiato rrweb e **ho deciso di non installarlo**, per tre motivi concreti:

1. **Playwright registra già.** Lo strumento che guida il browser produce da sé un video della pagina: aggiungere rrweb avrebbe significato due sistemi per fare la stessa cosa.
2. **Il DOM ricostruito non è garantito identico all'originale.** Font diversi, animazioni, `canvas`, video incorporati: la riproduzione può discostarsi da quello che l'utente ha visto davvero. Per un tutorial che deve mostrare **esattamente** ciò che si vede sullo schermo, è un rischio che non vale la pena correre.
3. **Avevo già un copione migliore.** Il percorso di visita viene salvato come file leggibile — con tempi, didascalie e selettori — e quel file è **già** la descrizione riproducibile della procedura. Un formato di registrazione in più non aggiungeva niente.

> *La lezione, che vale oltre questo caso: uno strumento buono non è uno strumento **da usare**. Prima di aggiungerne uno, la domanda giusta non è "funziona?", ma **"cosa mi dà che non ho già?"**. Se la risposta è "niente di decisivo", la scelta migliore è non installarlo — perché ogni pezzo in più è un pezzo da mantenere.*

rrweb resta però la scelta **giusta** in altri casi: quando il file deve essere piccolo, quando serve **cercare** dentro la registrazione, o quando si vuole che la riproduzione sia un documento interattivo e non un filmato. Sapere che esiste, e perché **non** usarlo qui, fa parte della cassetta degli attrezzi.

6 · GSAP: animazione per il web e per il video

GSAP è una libreria di animazione per il web. Nasce per far muovere elementi dentro una pagina, ma il suo uso più interessante per chi fa video è un altro: **animare una pagina HTML e poi riprenderla come fotogrammi di un video**.

Serve a: far comparire un titolo con una dissolvenza, far scorrere una scheda, far muovere un elemento al momento giusto, sincronizzare il testo con la voce.

Il vantaggio rispetto a un programma di montaggio è che **tutto è descritto in un file di testo**: la stessa animazione si può rigenerare identica domani, o su un altro computer, o con un testo diverso. Un video fatto così non è un file da ritoccare a mano: è un **progetto** che si riapre.

> ***Sulla licenza, il dato verificato.** Dal 2025, dopo l'acquisizione da parte di Webflow, GSAP è **gratuito anche per uso commerciale**, e sono diventati gratuiti anche i plugin che prima richiedevano l'abbonamento (SplitText, MorphSVG, ScrollSmoother e gli altri). Attenzione però a una distinzione che si fa spesso a rovescio: **gratuito non vuol dire open source**. GSAP ha una licenza **proprietaria** (la "GSAP Standard License"), non MIT né Apache: si può usare gratis, anche per lavoro, ma il codice non è aperto e non si possono rimuovere le note di copyright. Per un progetto dove conta la libertà della licenza, questo è un punto da tenere presente.*

7 · Meshy: oggetti 3D

Meshy genera **modelli tridimensionali** a partire da una descrizione testuale o da una fotografia. Si ottiene un oggetto 3D — una sedia, un pianeta, un ingranaggio — che si può girare, inserire in una scena, esportare come file `.glb` o `.obj`.

A cosa serve in un video? A tutto ciò che non si può filmare: un oggetto che non esiste, una ricostruzione, un'animazione tecnica. Molti strumenti di montaggio e di animazione 3D importano questi file e li fanno ruotare davanti alla telecamera.

Meshy è un servizio **nel cloud** (nessuna scheda grafica richiesta sul proprio computer: serve un browser e un account). Il piano gratuito dà **100 crediti al mese**, circa **dieci oggetti**, un compito alla volta, e permette di scaricare **al massimo dieci modelli al mese**. I piani a pagamento partono da circa **20 dollari al mese**.

Ma il dettaglio che conta davvero è un altro, ed è quello che si scopre sempre troppo tardi:

> ***Con il piano gratuito i modelli si scaricano con licenza CC BY 4.0**: si possono usare anche commercialmente, **ma citando la fonte**. Con i piani a pagamento la licenza diventa "privata", cioè uso commerciale pieno **senza obbligo di attribuzione**.*

Tradotto: il piano gratuito va benissimo per imparare e per progetti dove puoi citare Meshy; se il modello deve finire in un lavoro dove l'attribuzione non è possibile, serve il piano a pagamento. Non è un limite tecnico, è un limite di licenza — ed è meglio saperlo prima di costruirci sopra.

8 · Comfy.org e ComfyUI: immagini e video generati

Qui conviene fare subito una distinzione, perché è la fonte di quasi tutti gli equivoci.

ComfyUI è un programma **libero e gratuito**: si installa sul proprio computer e non manda niente a nessuno. È un'interfaccia a nodi — si collegano dei riquadri fra loro e si costruisce un procedimento — per generare immagini e video con i modelli di diffusione.

Comfy Cloud è il servizio **a pagamento** con lo stesso motore, che gira sui server di chi lo fornisce: si paga un abbonamento mensile con un sistema a crediti.

	ComfyUI (locale)	Comfy Cloud
costo	**gratuito**, licenza GPL-3.0	prova gratuita, poi abbonamento
dove gira	sul tuo computer	sui loro server
cosa serve	**una scheda grafica** (vedi sotto)	solo un browser e un account
riservatezza	tutto in casa	le immagini passano dal loro servizio

Il cloud parte con **cinque esecuzioni gratuite** su schede grafiche vere e un po' di crediti in omaggio, poi gli abbonamenti vanno da circa **20 dollari al mese** (35, 100 e oltre per i piani superiori), con un sistema a crediti: un video di cinque secondi costa, indicativamente, una decina

di crediti. Chi vuole solo l'API paga a ore di scheda grafica, da circa 3,50 a 11 dollari l'ora.

Il punto pratico, però, è la scheda grafica — e qui serve precisione, perché si sente dire "ComfyUI richiede una GPU" e non è esattamente così:

- La versione locale **parte anche senza scheda grafica**, usando solo il processore. È supportato ufficialmente. Ma su una macchina come questa (un i5 senza GPU dedicata) i tempi diventano **inutilizzabili**: si parla di ore per un'immagine, non di minuti.
- Quindi: "serve una GPU" è vero **per le prestazioni**, non in assoluto. Chi non ce l'ha e vuole usare ComfyUI davvero, oggi sceglie il cloud.

> *Il rovescio di questa medaglia è il capitolo 9: là l'intelligenza artificiale non **genera** le immagini, **scrive il programma che le disegna**. Un altro mestiere, con in cambio un controllo che qui manca.*

9 · Il video che è tutto codice: Lemo-Opuscar

Il capitolo precedente finisce con un modello che **genera** le immagini. Questo comincia dalla parte opposta, e la distinzione è il cuore di tutta la guida.

Un conto è chiedere a un'intelligenza artificiale di **inventare e generare** un video; un altro è chiederle di **scrivere il programma che lo costruisce**. Il risultato, in mano allo spettatore, può somigliarsi. Il processo no: sono due mestieri diversi, e la differenza si sente nel momento in cui vuoi cambiare qualcosa.

Due modi di fare un video

Col metodo generativo si descrive la scena — «un robot che cammina in una città futuristica sotto la pioggia, con luce da film» — e un modello addestrato su grandi quantità di filmati produce i fotogrammi, il movimento, le luci, gli effetti. È come avere una troupe al completo: regista, direttore della fotografia, attrezzisti. Il vantaggio è l'impatto: cose realistiche e scenografiche con poche parole. Il limite è il controllo: se vuoi spostare un oggetto di quindici pixel, o correggere una scritta senza toccare il resto, ti muovi nel vago. Rilanci e speri, e ogni nuova versione è **un'altra versione** — non la stessa con una virgola diversa.

Col metodo programmatico non si chiede il video: si chiede **il programma che lo disegna**. L'agente sceglie la sequenza delle scene, scrive il codice che traccia testi, forme, diagrammi e collegamenti, definisce animazioni, colori e tempi, e poi un motore grafico renderizza i fotogrammi, che vengono cuciti in un file video. Qui ogni elemento è **definito matematicamente**: è una coordinata, un colore, una durata. E proprio per questo si può cambiare.

> *La differenza, in una riga: nel primo caso l'intelligenza artificiale **immagina** il video; nel secondo **costruisce la macchina che lo produce**. E nel secondo, cambiare una cosa è cambiare una riga e rilanciare, con tutto il resto che resta identico a prima.*

Una precisazione che evita confusione

Attenzione a non mettere sullo stesso piano cose diverse. **Canvas, WebGL e SVG non sono modelli di intelligenza artificiale** e non generano video da soli: sono **tecnologie di disegno** — il foglio e la matita. Il video lo fa il codice che le usa.

Tecnologia	Cos'è	A cosa serve
Canvas 2D	il foglio da disegno del browser, guidato da JavaScript	testi, forme, grafici, animazioni piatte: infografiche animate, spiegazioni didattiche
WebGL (con three.js)	il motore grafico che sfrutta la scheda grafica	oggetti tridimensionali, particelle, simulazioni, ambienti
SVG	disegno vettoriale, ingrandibile senza perdita di qualità	diagrammi, icone, mappe, schemi
Remotion	un sistema che organizza un intero video scrivendolo come una pagina web	video didattici e presentazioni costruite a codice

Il chiarimento: le prime tre sono **come si disegna**; Remotion è **come si organizza un film** — scene, tempi, sincronizzazione — e sta un gradino più su.

Lemo-Opuscar: una libreria di stili, fatta di codice

Il metodo programmatico ha un esempio concreto, ed è la cosa che ho installato su questa macchina. **Lemo-Opuscar** è una libreria di **66 stili cinematografici per agenti**: non 66 filtri da applicare a un video, ma 66 **modi di girare**.

Ogni stile è un file di testo, `STYLE.md`, che fissa le **invarianti**: che aspetto ha, di quali materiali è fatto, che logica usa per il colore, che tipografia, che qualità di movimento, che grammatica di camera, che suono. Accanto al testo c'è una **demo costruita interamente in codice** — Canvas 2D, three.js, shader WebGL2 — che serve a mostrare quello stile in azione e a farne imparare la tecnica. Il testo da una parte, il film che lo prova dall'altra.

E qui arriva il punto, perché è lo stesso di tutto il capitolo:

> ***Nessun modello video.*** *Il film è* `window.render(t)`*: una pagina che sa disegnare se stessa all'istante* `t`*. Il render apre Chrome senza finestra, chiede i fotogrammi uno per uno e li cuce; l'audio è sintetizzato in casa e aggiunto con* `ffmpeg`*. Non c'è niente da addestrare e niente da generare: c'è codice che gira.*

In pratica, come funziona la filiera (è una filiera, non un singolo programma):

1. **si sceglie lo stile** e si scrive la sceneggiatura — la libreria la chiama *treatment*: cosa si vede, in che ordine, per quanto;
2. **si compone la linea del tempo unica** delle scene;
3. **si genera la voce** (voce sintetica, poi controllata da un riconoscitore che verifica che il parlato si dica davvero);
4. **si compone la musica** con librerie di campioni e sintesi, non con brani di terzi;
5. **si renderizza la parte visiva**: un browser per processo, i fotogrammi catturati e montati;

6. **si mixa e si cuce** tutto con `ffmpeg`.

Il bello è che **non serve una scheda grafica vera**: gira su questo portatile, con la grafica integrata, usando WebGL2; e se un effetto non regge c'è un ripiego in solo software, più lento ma affidabile. Per la parte vocale serve una sola accortezza: quel modulo è installato su un **Python 3.12 dedicato**, non su quello di sistema.

Dove sta, nel quadro

Lemo-Opuscar è **aperta** (il codice con licenza MIT, le demo con CC BY 4.0: si possono usare citando la fonte) e vive su un repository pubblico. È nata come **plugin di Claude Code** — uno di quei pacchetti che si agganciano all'agente per dargli una capacità in più — e io l'ho resa una **skill di Hermes**: è esattamente il meccanismo del capitolo 13, una procedura scritta una volta che l'agente carica quando serve.

Mettendo i tre modi di fare un video uno accanto all'altro, il quadro si chiude:

	IA generativa	Programmatico (codice)	Montaggio tradizionale
come nasce	il modello genera il video	il codice disegna i fotogrammi	si montano materiali già girati
controllo	indiretto, «per prompt»	**molto preciso**	manuale, a vista
realismo	elevato di suo	dipende da cosa disegni	dipende da cosa hai girato
testi e grafici	variabile	**molto preciso**	molto preciso
come si modifica	si riscrive il prompt	si cambia il codice	si sposta sulla linea del tempo
automazione	alta	**altissima**	variabile

E l'ultima proprietà è quella che vale per la didattica: siccome il video è codice, **il video è parametrico**. Si scrive **un** modello di film e se ne generano cento — cambiano argomento, dati, lingua, immagini e testi, mentre l'impianto resta lo stesso. Col metodo generativo questo non si fa: lì ogni video è un esemplare unico.

> *Non è una gara fra i tre metodi: si combinano. Le scene cinematografiche dove servono, i grafici e gli schemi fatti a codice dove serve la precisione, il montaggio a chiudere il tutto. La filiera del capitolo «Come si combinano» è esattamente questo, e un video di formazione serio li usa tutti e tre.*

10 · AnyMP4 / AnyRec: conversione e montaggio

Sono programmi con l'interfaccia grafica per convertire video, registrarne lo schermo, scaricare da internet, montare. Fanno il loro lavoro, e a molti risultano più comodi di una riga di comando.

Ma qui bisogna essere chiari, perché è esattamente il tipo di strumento che in un elenco "gratuito" non dovrebbe stare senza avvertenza:

> ***Sono prodotti commerciali**, e i numeri lo dicono: **AnyMP4** costa intorno ai 12 dollari al mese (o circa 100 per una licenza valida su cinque computer), **AnyRec** circa 14,50 al mese oppure 55 per una licenza a vita. Il programma da tavolo gratuito è una **versione di prova**: serve a provare, non a produrre.*

C'è però una parte gratuita che vale la pena conoscere: entrambi offrono un **registratore di schermo *online*, dentro il browser**, senza installare niente e senza account. Le differenze contano:

	registratore online	limiti
AnyMP4	gratuito, dichiarato **senza filigrana**	massimo 24 fotogrammi al secondo (contro 60), esporta solo in WMV e MP4, meno strumenti, niente registrazione programmata
AnyRec	gratuito, senza limite di durata, senza account	**aggiunge una filigrana** sull'esportazione

Quindi, se serve una registrazione veloce e senza installare nulla, **quello di AnyMP4 è la scelta migliore** — purché 24 fotogrammi al secondo bastino (per un tutorial bastano; per un videogioco no).

Perché allora stanno in questa guida? Perché sono un **punto di partenza comodo** per chi non ha mai toccato ffmpeg: si impara cosa serve fare (convertire, tagliare, unire) con i pulsanti, e poi — quando si capisce che la stessa cosa si fa con una riga — si passa a ffmpeg, che è gratis per sempre e non ha filigrane.

Il criterio, che vale in generale

Quando un programma gratuito chiede di pagare, guarda **cosa** ti sta vendendo:

- se ti vende **l'interfaccia** (pulsanti comodi attorno a un motore gratuito), allora lo stesso lavoro si può fare con lo strumento libero e un po' di pratica;
- se ti vende **il motore** (un modello di intelligenza artificiale che gira sui loro server), allora lì il pagamento è reale, e conviene valutare quanto lo usi.

11 · Pollinations: contenuti generati via API

Pollinations è un servizio che genera **immagini, testo, audio e video** con un'interfaccia molto semplice: spesso basta un indirizzo web. È la via più diretta per aggiungere un'immagine o una voce a un progetto senza installare nulla.

Cosa funziona senza chiave

Questo è il punto interessante, ed è **verificato**:

- **le immagini si generano senza chiave**: basta aprire un indirizzo con la descrizione dentro, e si riceve un'immagine;
- **il catalogo dei modelli è pubblico**: si può leggere quali modelli esistono (sono centinaia), quanto costano e se sono gratuiti;
- **il testo dipende dall'indirizzo che usi**, e questa è la sfumatura che ho dovuto misurare per capirla:
- l'**indirizzo semplice** (`text.pollinations.ai/…`) risponde **anche senza chiave**;
- l'**API vera** (`/v1/chat/completions`), quella che si usa dal proprio programma, **pretende una chiave** e senza risponde "rifiutato".

Non si può quindi dire "è gratis" in blocco: dipende da **cosa chiedi** e da **come lo chiedi**. La regola è controllare il catalogo — che è pubblico apposta — invece di fidarsi di un'etichetta.

> *Un'ultima distinzione, che vale per tutto questo capitolo: il **codice** di Pollinations è aperto (licenza MIT), ma il **servizio** è a crediti. Sono due cose diverse: puoi leggere e riusare il programma, non il servizio che qualcun altro paga.*

La chiave senza farla passare da nessuna parte

C'è un modo per ottenere una chiave **senza incollarla in una chat, in un comando o in un file di configurazione**, e si chiama **device flow**: il servizio mostra un indirizzo e un codice, **l'utente apre quell'indirizzo nel proprio browser e approva**, e il programma riceve la chiave da solo. È lo stesso meccanismo con cui si autorizza un televisore a usare un account di streaming.

Il vantaggio è che **la credenziale non passa mai da chi la deve usare**: nessuno la scrive, nessuno la legge, nessuno la salva in un posto sbagliato.

12 · La tecnica: far entrare un agente nei propri profili

Questo è il capitolo per cui vale la pena leggere la guida, perché cambia il modo di lavorare più di qualunque strumento dell'elenco.

Il problema

Molte cose che vorremmo far fare a un assistente automatico — aprire un servizio, guardare un'interfaccia, esportare dei dati, registrare una procedura — richiedono un **accesso**: un nome utente e una password, o una sessione già aperta.

La soluzione ingenua è dare all'automatismo le credenziali. **È sbagliata**, per tre motivi:

1. la password finisce in un programma, in un file, o in un registro di log;
2. non funziona con i servizi che richiedono un secondo fattore di sicurezza;

3. e nei casi peggiori il servizio **blocca** l'accesso, perché riconosce che il browser non è guidato da una persona.

La tecnica, in tre passi

Si chiama **accesso delegato**: l'automatismo non ha le credenziali, e non gli servono.

1. **L'automatismo apre una finestra di navigazione propria** — con un profilo dedicato, mai il profilo personale di chi lo usa — e si presenta al sito come un browser normale.
2. **La persona fa l'accesso in quella finestra**, una volta sola, come farebbe sempre. La password non passa da nessun'altra parte: resta fra la persona e il sito.
3. **L'automatismo si accorge da solo che l'accesso è stato fatto**, salva la sessione e **riprende da dove era rimasto**. Da lì in avanti entra già autenticato, senza chiedere più niente.

È **la stessa idea del device flow** del capitolo precedente, applicata non a una chiave ma a una sessione di navigazione. Il principio comune è: *chi ha le credenziali le tiene, chi deve usarle riceve solo il permesso*.

I tre dettagli che fanno la differenza

Questi sono i punti in cui la tecnica funziona o non funziona, e li ho verificati uno per uno.

Il browser non deve dichiararsi automatizzato

Di norma gli strumenti di automazione si annunciano come tali, e i servizi più attenti rispondono *«questo browser potrebbe non essere sicuro»* **prima** di mostrare il modulo di accesso. Non è un blocco sulla password: è un blocco sul **browser**.

Togliendo quel segnale, la pagina di accesso torna normale. È una riga di configurazione, e cambia tutto: senza, il servizio non ti fa nemmeno provare.

Si registra quello che si vede

Vale per la registrazione dello schermo, e vale qui: se la finestra non è davanti alle altre, nel video — o nella procedura — finisce qualcosa d'altro. Prima di partire, la finestra va portata in primo piano **e verificato** che ci sia davvero. Il sistema non lo concede a chiunque: servono un paio di accorgimenti tecnici che ho dovuto scoprire sbagliando.

L'automatismo deve capire da solo quando ha finito

Non si può chiedere alla persona di premere "ho finito", perché è proprio quello che si voleva evitare. L'automatismo aspetta, e a un certo punto si accorge che l'accesso è avvenuto — perché la pagina non chiede più di entrare **e** perché i dati di sessione sono aumentati. Servono **due segnali insieme**, e serve una **attesa minima**: un accesso fatto da una persona non richiede tre secondi, e se si controlla troppo presto si conclude per sbaglio che è già tutto a posto.

Perché conviene, in una riga

Perché **l'automatismo non custodisce nessuna credenziale**, e questo risolve insieme il problema della sicurezza, quello dei servizi con doppio fattore, e quello dei servizi che bloccano i

browser automatici. La persona resta la sola a conoscere la propria password, e l'automatismo lavora con il permesso che gli è stato dato.

La regola d'oro

Non lasciare mai un vicolo cieco. Se l'accesso automatico non riesce, il messaggio deve offrire **almeno due strade**, e almeno una deve essere sempre percorribile — per esempio: *«fai tu questa parte nel tuo browser, dove sei già dentro, e passami il risultato: il resto lo faccio lo stesso»*.

Un errore che dice soltanto "non posso" fa perdere tempo a chi lo legge, e fa sembrare rotto uno strumento che invece sta funzionando: ha semplicemente incontrato un caso che non può risolvere da solo.

13 · Le skill: procedure scritte una volta

Fin qui la guida ha parlato di **strumenti**: cose che si installano e si usano. Quest'ultima parte parla di un'altra cosa, che non si scarica e non si paga: il **modo di lavorare**.

Una **skill** è una procedura scritta una volta e riusata sempre. Non è un manuale da leggere dall'inizio: è un foglio di istruzioni che l'agente **carica solo quando serve**, cioè quando il lavoro somiglia a quello descritto. È il motivo per cui lo stesso agente sa fare cose molto diverse — scrivere un articolo, montare un video, pubblicare un sito — senza diventare confuso: non tiene tutto in testa insieme, prende il foglio giusto al momento giusto.

> *La differenza con un manuale è tutta qui: un manuale lo leggi **prima**, una skill la legge **l'agente**, e solo se il lavoro la riguarda.*

Le skill che contano, per area

Queste sono le vitali in questa installazione, con il criterio: **quelle che cambiano davvero il risultato**. I nomi sono quelli tecnici, perché sono quelli che servono per usarle.

Area	Skill	A cosa servono
Costruire app locali	`local-first-webapp-stdlib`, `windows-background-services`, `local-web-app-e2e-testing`, `headless-webapp-verification`	la convenzione di casa: nessun framework, `http.server`, configurazione in un file JSON, avvio con un `.bat`, il tasto ■ Guida dentro l'app

Area	Skill	A cosa servono
Video e multimedia	`narrated-tutorial-video`, `hyperframes-video`, `hyperframes-cli`, `navigazione-registrazione-video`, `accesso-delegato-utente`, `local-transcription`, `avatar-voce-clonata`	dalla registrazione muta al film con voce clonata; e l'accesso ai propri profili **senza dare la password**
Scrivere e pubblicare	`crea-articolo-seo-geo`, `crea-articolo-umanesimo-digitale`, `audit-seo-geo-francobagaglia`, `humanizer`, `grounded-citations`, `formal-document-production`, `pdf-book-authoring`	i due siti, l'audit obbligatorio prima di pubblicare, i documenti formali, i PDF impaginati
Programmare con gli agenti	`hermes-agent`, `launch-subagent`, `claude-code`, `codex`, `git-worktree`, `tdd`, `systematic-debugging`, `verifying-technical-claims`	delegare il codice a più agenti in parallelo, e **non credere a un referto senza verificarlo**
Scrivere skill	`effective-agent-skills`, `writing-for-agents`	come si scrive una skill che funziona: cosa metterci, cosa lasciar fuori
Conoscenza e studio	`obsidian`, `second-brain-vault-ingest`, `hermes-wiki`, `deep-research`, `web-verified-search`	il vault, l'ingest dei documenti, la ricerca con fonti verificabili
Automazione	`hermes-cron-management`, `hermes-bot-profiles`, `email-inbox-triage`, `moodle`	lavori programmati, bot, posta, i corsi sulla piattaforma
Sicurezza	`third-party-package-vetting`, `risky-changes`, `global-agent-guardrails`	prima di installare, prima di pubblicare, prima di un comando pericoloso

Le regole che rendono utile una skill

1. **Una skill dice cosa fare, non cosa sapere.** Se non cambia il modo in cui il lavoro viene eseguito, non è una skill: è un appunto.
2. **Meglio poche e vere che molte e generiche.** Una skill che dice "scrivi bene" non serve a niente; una che dice "EN senza trattini lunghi, audit obbligatorio prima di pubblicare" cambia il risultato.

3. **Si scrive** dopo **aver sbagliato.** Il valore di una skill sta negli errori che evita — quelli che sono costati tempo. Le trappole trovate nei collaudi valgono più dei consigli generali.
4. **Si aggiorna quando si scopre qualcosa.** Una skill che resta ferma mentre il lavoro cambia diventa un ostacolo, non un aiuto.

14 · I connettori: dare le mani all'agente

Saper scrivere non basta: per agire in un servizio esterno — leggere la posta, creare un evento, aprire un progetto — l'agente ha bisogno di un **connettore**.

Il connettore è l'autorizzazione con cui l'agente entra in un servizio **per conto tuo**, e la regola è sempre la stessa, in ogni caso:

> ***L'utente autorizza, l'agente non vede mai la password.*** *Non è una gentilezza: è ciò che rende possibile collegare i propri account a un automatismo senza consegnargli le chiavi di casa.*

Molti di questi connettori usano un **protocollo standard** (si chiama MCP), che è il motivo per cui si aggiungono e si tolgono senza cambiare l'agente.

Quelli disponibili qui

Ambito	Connettori
Posta e calendario	**Gmail** — e con esso **Google Calendar**: lo stesso connettore crea eventi, non solo legge la posta
Grafica	**Canva** — già connesso e attivo
Documenti e progetti	Notion, Confluence, Airtable, Linear, Jira, Asana, ClickUp, Box
Il più ampio: Composio	collega **oltre 500 servizi**: Google Workspace (Gmail, Fogli, Drive, Calendar), Microsoft (Outlook, Teams), Slack, GitHub, Figma, X, Instagram, TikTok, Meta Ads
Pagamenti e commercio	Stripe, Shopify, Coinbase, Plaid, Ramp
Sviluppo	Vercel, Sentry, Railway, Cloudflare, Datadog, PostHog, Neon, Supabase
Vita quotidiana	Todoist, Calendly, Discord

In questa installazione sono abilitati **55 connettori**.

La regola che conta

Se ne connette uno quando serve, non tutti insieme. Ogni connettore è un permesso in più, e i permessi si danno uno alla volta, quando il lavoro li richiede. È la stessa logica dell'accesso delegato del capitolo 12: chi ha le credenziali le tiene, chi deve usarle riceve **solo il permesso che**

gli serve — e il permesso si può togliere.

15 · I progetti

Questa parte non parla di strumenti ma di **cose costruite**: i repository e i cantieri che stanno su questa macchina. Serve a due scopi — avere un indice di cosa c'è, e capire **come** questi strumenti sono stati usati insieme.

I tre repository di riferimento

Repository	Visibilità	Cosa contiene
guida-multimedia	**pubblico**	questa guida: il PDF impaginato, il testo sorgente in Markdown, il generatore del PDF e i controlli automatici, più il documento con le fonti verificate
creazione-video	privato	il materiale dei due corsi: `chiavi_di_lettura.md` (gli script, la fonte unica), `indice_moduli.json` (i 33 moduli), `IMPIANTO.md` (il verbale delle decisioni). **Solo testo**: i video restano sul PC
regia-tutorial	privato	il codice dell'app che monta i video: da una registrazione muta e due testi produce il film con voce clonata, musica e dissolvenze

I cantieri: cosa sono, cosa contengono

1 · Corso Online Sintetico

Il corso Moodle **607** dell'Academy SSML — «Aggiornamento all'Intelligenza Artificiale» — in **15 unità da 60 minuti**, ognuna con 20 minuti di testo, 30 di pratica e 10 di verifica. È la versione sintetica, per pubblico ampio, del più grande progetto «Corso Online».

Contiene il piano e lo stato del corso, i testi delle unità con quiz, slide e fonti, gli script Python che **pubblicano su Moodle** via API, e i video.

Tecnologie: Markdown trasformato in HTML da script Python, pubblicazione REST su Moodle, voce sintetica per i video.

Stato: in produzione e in aggiornamento continuo — i moduli sono online sul corso 607.

2 · AI Essential 2 SSML

L'officina che costruisce **due corsi gemelli** spezzando e riusando il Corso Online Sintetico: i corsi universitari Moodle **579** («AI Essentials: informatica, modelli e applicazioni») e **580** («AI Essentials II: dal chatbot al collaboratore»).

Contiene il compositore deterministico, le due cartelle dei corsi (11-12 moduli ciascuno, con quiz e slide Marp), gli indici, i rapporti di composizione e i materiali comuni.

Tecnologie: testi Markdown, figure generate da script, slide Marp, quiz nativi Moodle caricati via API.

Stato: **materiale didattico già pubblicato** — entrambi i corsi caricati a ottobre 2026. Non un prototipo.

3 · REA

Non è un progetto mio: è il **clone locale di un progetto open source di terzi** (`morluto/rea`, licenza MIT), con in più un dossier di studio che ho scritto. REA è un server che dà a un agente la capacità di **analizzare software di cui non ha il sorgente** — binari, applicazioni Electron, .NET, Android, firmware — restituendo ogni conclusione **con le prove** e i limiti.

Contiene il codice TypeScript (oltre 1.200 file), i test, la documentazione tradotta in venti lingue, e il dossier con l'architettura, il catalogo dei comandi e le verifiche fatte davvero su questa macchina.

Stato: installato e funzionante, con prove reali su eseguibili di sistema e su un'app Electron.

4 · Deep Dog

Un piccolo laboratorio attorno a un motore di **ricerca approfondita** di terzi: da una domanda produce un **rapporto con citazioni**, con un supervisore che pianifica, dei sub-agent che cercano e le fonti che vengono validate.

Il motore non è mio; i file miei sono tre: un lanciatore, un'estensione che aggiunge un motore di ricerca diverso da quelli previsti, e un file per il doppio clic.

Stato: funzionante e già usato — c'è un rapporto reale di 96 KB sull'evoluzione dei mestieri della traduzione.

5 · Modello Delega

Il progetto che spiega e mette in pratica **come si delega una piattaforma a un assistente**: un lavoro che si faceva a mano diventa una frase, l'assistente ricognisce, agisce e **riporta la prova**. Il modello è fatto di sei domande — canale e credenziale, sonda dei permessi, identificativo dell'unità di lavoro, falsi successi, come si verifica, cosa non si disfa.

Contiene il modello come procedura per l'assistente, l'esempio reale su Moodle (autoconfigurante da chiavetta), le tre forme di squadra di agenti e una guida per docenti in sette capitoli.

Stato: finito e **collaudato dal vivo** — l'esempio Moodle ha prodotto esiti verificati, e la prova su posta elettronica ha insegnato la lezione sui canali che coprono solo una parte del lavoro.

6 · Menu App loopback

Una pagina sola con tre ingressi che apre le applicazioni locali al clic; **se l'app è spenta la avvia** e aspetta che risponda prima di aprire il browser. Serve perché una pagina aperta come file non può lanciare un processo.

Contiene la configurazione con le tre voci, il server, la pagina con i pallini di stato e i launcher.

Tecnologie: Python di sola libreria standard, in ascolto **solo su indirizzo locale** e che rifiuta richieste non locali.

Stato: funzionante e completo. Non reimplementa i launcher dei progetti: li invoca come farebbe l'utente.

7 · SSLM Second Brain

La piattaforma locale che trasforma PDF e note in **note di studio collegate** dentro un vault Obsidian — una nota centrale con indice e rimandi, più note tematiche — e poi le fa **ripassare a intervalli crescenti**, con quiz valutati dall'AI, interrogazione delle note, pacco di schede per il ripasso e generazione di immagini e video.

Contiene il backend in Python, l'interfaccia, un vault di 108 note, i 35 capitoli del volume «L'Esoscheletro Cognitivo», le guide per i docenti e gli script di verifica.

Tecnologie: Python con **sole due dipendenze** (per leggere i PDF), interfaccia in JavaScript puro, **nessun database** (lo stato sta nel frontmatter delle note), AI via OpenRouter con due modelli — uno per il testo, uno per audio e video.

Stato: funzionante e **in esecuzione adesso**, versione 3.72. È insieme prodotto in uso e corpus didattico.

8 · Mio_Esoscheletro_Z

Applicazione web per la gestione della conoscenza personale secondo il metodo **PARA** e la piramide **DIKW**: note con metadati, elaborazione AI delle note grezze, chat socratica, grafo della conoscenza, flashcard con ripetizione spaziata, import/export di progetti.

Contiene circa venti rotte di programmazione, quattordici componenti di dominio (dashboard, piramide, grafo, chat, editor), un vault di 35 note e il grafo.

Tecnologie: **Next.js 16 con TypeScript**, Tailwind, grafo con D3.js, dati su file Markdown, AI tramite un servizio esterno.

Stato: **prototipo sperimentato e superato**, non un'app in uso. Il progetto è stato costruito in un ambiente cloud e poi copiato qui; localmente non è avviabile così com'è. Il suo ruolo è stato quello del **banco di prova**: molte idee sono poi confluite in SSLM Second Brain, che è la versione matura e funzionante.

9 · AGENT OS

Dashboard locale per **censire e pilotare agenti da riga di comando di fornitori diversi**, con il confronto affiancato dello stesso prompt su più agenti e con **terminali veri** dentro la pagina.

Contiene il server in Python di sola libreria standard, il gestore dei processi, il catalogo degli agenti, la scoperta automatica di quelli installati, una libreria di **50 prove in 16 categorie** e i collaudi automatici.

Tecnologie: Python con una sola dipendenza per i terminali, interfaccia senza framework, funziona **offline**.

Stato: funzionante e collaudato (26 commit).

10 · Creazione Testo Ultimate

Generatore locale di contenuti: articolo (per argomento, rivisitazione o da un link), accompagnamento per i social, domande frequenti, pacchetto per i motori di ricerca, griglie di schede, hashtag per piattaforma, titoli alternativi, traduzione italiano/inglese e «umanizzazione» del testo.

Due pagine — lo studio e l'officina — con backend in Python di libreria standard che interroga ricerca web e Wikipedia (la ricerca non può stare nel browser, per una restrizione di sicurezza dei browser stessi), e i modelli via OpenRouter.

Stato: **il progetto più avanzato e più attivo** (35 commit, backend alla versione 2.12). Ha assorbito ed esteso il precedente «Crea Post».

11 · Hermes Logos

Piattaforma **local-first** per la gestione della conoscenza, il ripasso spaziato e l'automazione quotidiana, in due aree: un gestore di attività (posta multi-casella, rassegna dal web, diario di apprendimento, notizie, lavori programmati) e l'esoscheletro (second brain su vault Obsidian).

Contiene il motore in Python (oltre 5.000 righe), la dashboard, l'amministratore dei lavori programmati, il plugin per Obsidian e le guide.

Tecnologie: Python con un framework web, interfaccia in HTML e JavaScript, bot di messaggistica, sincronizzazione **a una via** verso il server remoto (che è vetrina, non seconda fonte).

Stato: **in esercizio quotidiano** (154 commit).

12 · Comune Demo Console

«Mission Control»: una console locale per **coordinare più agenti da un'unica interfaccia** durante il corso per gli impiegati comunali — con tre esecutori diversi collegati insieme.

Contiene il server, l'interfaccia, il catalogo dei casi d'uso per la pubblica amministrazione (posta certificata, anagrafe, edilizia, intermediazione linguistica), i client separati per ogni esecutore, diciassette profili di agenti e **dati fittizi**.

Tecnologie: Python di libreria standard, interfaccia senza framework, un launcher che classifica chi occupa le porte e termina **solo** i processi del progetto (mai i servizi condivisi con la produzione).

Stato: pronto e **usato in aula** (22 commit).

13 · Agenti per tutti

Kit didattico — documenti, non software — per il corso «2026: l'anno degli agenti»: tre ore per gli impiegati dei Comuni di **Allumiere e Tolfa**, più un modulo universitario su studio e intermediazione linguistica.

Contiene il manuale operativo di 38 KB, la teoria delle cinque ingegnerie (dal prompt al grafo), quattro casi studio comunali, il piano d'aula minuto per minuto, la parte su governance ed etica nella pubblica amministrazione, una raccolta di 40 istruzioni pronte e i materiali di supporto.

Tecnologie: puro Markdown, con dimostrazioni dal vivo.

Stato: **consegnabile e chiuso** (12 commit). Lo standard di ricerca web verificata nato qui è stato poi adottato dagli altri progetti.

14 · Crea Post

Web app locale che genera contenuti per i social: titolo e corpo del post, abstract, tweet, hashtag, domande frequenti, link suggeriti e versione umanizzata del testo.

Tecnologie: HTML, CSS e JavaScript senza framework, generazione via OpenRouter (migrata da un altro fornitore), cronologia nell'interfaccia.

Stato: **il più vecchio della lista** e non più sviluppato: è proseguito nei due successori, «Creazione Testo Ultimate» e «Crea Post Abacus».

Gli altri repository

Oltre ai cantieri qui sopra, la cassetta degli attrezzi contiene altri repository, più piccoli o più specialistici:

Repository	Cosa contiene
corso-secondo-cervello-agentico	il corso online «Il secondo cervello agentico», 15 moduli
corso-agenti-ssml · **corso-agenti-sslm**	il corso sugli agenti AI per docenti universitari, nelle due edizioni
pacchetto-delega	il pacchetto d'installazione: il Cervello, i bot, il modello di delega
agent-os	la dashboard per censire e pilotare agenti (versione condivisa)
private-skills	le skill condivise fra i miei agenti, in un'unica cartella canonica
hermes_logos	la piattaforma Hermes Logos (versione condivisa)
anki-bot	l'agente che prepara le schede di ripasso per il gruppo di studio

Repository	Cosa contiene
creazione-testo-ultimate	il generatore di contenuti (versione condivisa)
menu-app-loopback	il menu delle applicazioni locali (versione condivisa)
ssml-second-brain	la piattaforma di studio (versione condivisa)
ai-essentials-2-ssml	l'officina dei due corsi gemelli (versione condivisa)
nanoclaw-setup	l'assistente NanoClaw sul server remoto: guida operativa
comune-demo-console	la console di coordinamento agenti (versione condivisa)
agenti-per-tutti	il kit didattico per gli impiegati comunali (versione condivisa)
SkillHub · dashboard	raccolta di skill e pannello personale
tts-italiano-premium	sintesi vocale in italiano
formazione	materiale dell'Accademia Formazione (**pubblico**)
umanesimodigitale	il motore di ricerca del sito UmanesimoDigitale.info (**pubblico**)
francobagaglia	il sito personale

Cosa si vede, guardando l'insieme

Tre cose, che valgono più di un elenco.

La prima: quasi tutto è **locale e senza dipendenze pesanti**. La maggior parte dei progetti usa Python di libreria standard e JavaScript senza framework, e **non ha un database** — lo stato sta in file leggibili. È una scelta precisa: significa che un progetto si può riaprire fra due anni e capirlo, senza ricostruire un ambiente.

La seconda: i progetti **si sono succeduti**, non moltiplicati. «Crea Post» è diventato «Creazione Testo Ultimate». «Mio_Esoscheletro_Z», costruito in ambiente cloud, è stato il banco di prova da cui è nato «SSLM Second Brain», che è la versione matura e in uso. Questo è il modo giusto di lavorare: il prototipo **serve** a scoprire cosa non funziona, e poi si butta.

La terza: c'è un filo che li lega tutti, ed è lo stesso di questa guida — **la procedura scritta una volta**. Il modello di delega, le skill, gli script che pubblicano i corsi, il generatore di questa guida: sono tutti modi diversi di dire la stessa cosa. Non eseguire un lavoro a mano, ma **descriverlo**, perché la volta dopo si esegua da solo.

16 · Portare tutto su un altro computer

Un'ultima cosa, che non è multimedia ma è la domanda che viene subito dopo: **se cambio computer, cosa mi porto dietro?**

La risposta è più semplice di quanto sembri, perché l'installazione **non è sparsa**: vive in **una sola cartella**.

Una cartella sola

Tutto sta in `%LOCALAPPDATA%\hermes\` — cioè, su Windows, `C:\Users\<nome>\AppData\Local\hermes\`. Memoria, skill, connettori, lavori programmati, configurazione, credenziali: **tutto lì dentro**. Non ci sono pezzi nascosti altrove.

Si sceglie quanto portare

Questa è la parte utile: **non si è obbligati a portare tutto**. Si sceglie per livelli, e la differenza è grande.

Livello	Cosa si aggiunge	Peso	Cosa si ritrova
1	memoria, configurazione, personalità, credenziali, **autorizzazioni dei connettori**, lavori programmati, plugin	**~1 MB**	chi sei, come lavori, i connettori già autorizzati
2	**+ le skill**	**+110 MB**	**l'installazione torna a essere la tua**: senza skill l'agente sa ragionare, con le skill sa *come fai tu le cose*
3	+ la cronologia delle conversazioni	+75 MB	rileggere le chat passate
4	+ il database completo	**+1,8 GB**	la memoria storica integrale

Il livello 2 è quello che consiglio: circa **110 MB**, sta in una chiavetta, e riporta tutto quello che conta. Il livello 4 pesa quasi due gigabyte e serve solo se vuoi davvero lo storico integrale: per il resto è zavorra.

> ***I 110 MB sono quasi tutti le skill.*** *Il resto — memoria, configurazione, chiavi, autorizzazioni, lavori programmati — sta in* ***meno di 3 MB****. Il peso delle skill non è testo: sono le immagini e i file di supporto che alcune si portano dietro.*

C'è anche una cartella da NON portare

Nella cartella dell'installazione ce n'è una che sembra importante e non lo è: si chiama `tools`, pesa **1,8 GB** e contiene i **programmi scaricati** dal sistema per lavorare — il browser per l'automazione, `ffmpeg`, `node`, `python`, `git`, gli strumenti di ricerca nel testo.

Non va copiata: sono cose che il sistema **riscarica da solo** sul computer nuovo, la prima volta che servono. Copiarle significherebbe portarsi dietro quasi due gigabyte di file che verrebbero comunque riletti e rigenerati — e che, se il computer nuovo ha una versione diversa di Windows, potrebbero persino non funzionare.

È lo stesso errore in cui si cade con i progetti: **si copia ciò che non si sa ricostruire, non ciò che si sa riscaricare.**

> *La voce che quasi tutti dimenticano sono le **autorizzazioni dei connettori**: non stanno nel file di configurazione, ma in una cartella a parte (`mcp-tokens`). Se non la porti, i connettori vanno **riconnessi a mano, uno per uno** — anche avendo copiato tutto il resto.*

Come si fa, in pratica

Il backup e il ripristino sono due comandi, e si danno a due file già pronti: uno copia (fermando prima il servizio, ed escludendo cache, registri e temporanei), l'altro ricopia sul computer nuovo.

L'ordine è questo:

1. **Sul computer vecchio**: si lancia il backup, indicando dove salvare (una chiavetta, un disco esterno).
2. **Sul computer nuovo**: si installa Hermes, **si chiude**, e si lancia il ripristino.
3. Si riapre: memoria, skill, connettori, lavori programmati e configurazione sono già lì.

Le tre precauzioni che contano

1. **Il backup contiene segreti** — credenziali e autorizzazioni. Non va su un servizio non cifrato, e la chiavetta va trattata come una chiave di casa: chi la trova, entra.
2. **Il nome utente cambia.** Dentro la configurazione ci sono percorsi assoluti del tipo `C:\Users\<nome>\...`: vanno aggiornati. È la causa numero uno dei «non funziona niente» dopo un trasferimento, ed è una cosa da fare subito, non da scoprire dopo.
3. **I file di avvio vanno scritti in formato Windows** (a capo CRLF, caratteri semplici senza accenti): altrimenti il sistema non li esegue, senza spiegare perché.

Il principio, che è lo stesso di tutta la guida

Vale anche qui la regola che tiene insieme ogni capitolo: **la cosa importante non sono i file, è il modo di lavorare che contengono**.

Portarsi dietro millequattrocento skill non serve perché sono tante, ma perché ognuna è **una procedura che qualcuno ha già faticato a scoprire** — le trappole, le scorciatoie, gli errori da non ripetere. È quello che un'installazione nuova non ha, e che ricostruire a mano costerebbe mesi.

17 · OpenMausBot: mettere in fila gli agenti che già paghi

Prima di raccontarlo, la differenza che conta — perché è la stessa domanda che si fa chi usa questa guida.

Hermes è un agente; OpenMausBot è un orchestrabot. Hermes porta **il suo** motore, i suoi strumenti, la sua memoria e le sue procedure: con una chiave OpenRouter fa tutto da sé. OpenMausBot **non ha un agente proprio**: prende i programmi da riga di comando che hai già installato — `claude`, `codex`, `grok` — e ti dà una bella interfaccia per parlarci. **Senza uno di quegli abbonamenti, OpenMausBot non fa niente.**

	Hermes	OpenMausBot
Che cos'è	agente completo, con harness proprio	orchestratore di CLI esistenti
Chi pensa	il provider che scegli (OpenRouter)	i CLI che hai già: Claude, Codex, Grok
Interfaccia	app desktop, riga di comando, gateway per i bot	**chat a contatti**: ogni bot è un contatto
Memoria	sua, persistente fra le sessioni	**del singolo bot**, non condivisa fra loro
Skill	**1.428 file**, sue	nessuna propria: quelle dei CLI
Estensione	skill, plugin, connettori MCP	**un file** per aggiungere un fornitore
Permessi	delegati al modello	**broker**: ogni azione rischiosa è una carta *approva/nega*
Computer	la tua macchina (e un server)	cloud Linux, VM locale, o il tuo PC
Costo	la tua chiave OpenRouter	i tuoi abbonamenti a Claude o Codex

L'idea

Un'app di messaggistica dove **ogni contatto è un agente**. Non una finestra di chat sola: una colonna di bot, ognuno con il suo filo, la sua memoria, il suo modello, il suo computer e le sue app collegate. Passi da un bot all'altro come passi da una conversazione all'altra in un'app di messaggi.

È nato come **alternativa libera** a prodotti chiusi dello stesso tipo — Grok Bot di xAI, Muse, dots, Cue — ed è **open source** (licenza Apache-2.0), scritto in TypeScript. Non è affiliato a nessuno di quei produttori: il nome lo dichiara apertamente.

Come è fatto dentro: due processi

È la parte che vale la pena capire, perché spiega tutto il resto.

Il primo processo è l'app. È volutamente **stupida**: non gestisce da sé nessun collegamento ai CLI, non conserva stato, non parla con nessun servizio esterno. Manda comandi con una richiesta web e ascolta **un solo flusso continuo di aggiornamenti** che torna indietro. Tutto quello che vedi sullo schermo — un messaggio che compare, uno strumento che viene usato — passa da quel flusso unico.

Il secondo processo è il server del harness, e fa tre mestieri:

1. **Il registro dei piloti.** Un *pilota* per ogni fornitore supportato, e ognuno sa parlare la lingua di quel fornitore. Serve perché Claude e Codex **non comunicano allo stesso modo**: uno usa un formato a flusso, l'altro un protocollo a chiamate remote. Invece di costringere l'interfaccia a capire entrambi, il server **traduce tutto in un formato unico** appena arriva. È il pezzo tecnicamente più interessante: aggiungere un fornitore nuovo non richiede riscrivere l'app, basta **un file**.
2. **Il bus degli eventi**, che smista quel flusso normalizzato, e lo **scrive su disco** conversazione per conversazione: niente va perso.
3. **Il broker dei permessi**: intercetta le azioni rischiose — comandi di shell, modifiche di file — e le trasforma in **carte di approvazione** prima che succeda qualcosa. Sta sul server, non nell'app, così la logica è una sola per tutti i bot e tutti i fornitori.

Tutto gira **sulla tua macchina**: il server ascolta su `127.0.0.1` e i dati — trascrizioni, chiavi, registri — stanno in una cartella nascosta del tuo computer, non su un cloud.

Le tre cose che fa meglio

1. **I permessi.** Ogni azione rischiosa diventa una carta con *approva* e *nega* **dentro la chat**. Non è un'impostazione sepolta: è il modo normale di lavorare. È l'opposto dell'agente che si fida e va avanti.
2. **I bot si parlano.** Puoi metterne uno come capo e gli altri come specialisti — posta, spese, reclutamento, correzioni — e si scambiano contesto **da soli**, in chat di gruppo, senza che tu faccia da tramite. **Attenzione a cosa si condivide e cosa no**: in una stanza i bot vedono **la stessa trascrizione** e le **istruzioni condivise**, ma **non hanno una memoria comune**. Ognuno ha la sua — la memoria, il modello e il computer sono **del singolo bot**. Non è un agente con più voci: sono **agenti distinti che si passano il contesto nella stanza**.
3. **Vedere il lavoro.** Ogni bot può avere un computer con **anteprima dal vivo**: guardi cosa sta facendo mentre lo fa, invece di leggere un riassunto dopo.

Le cose che gli mancano, rispetto a Hermes

1. **Non ricorda fra le sessioni — e non ha una memoria comune.** La memoria è **del singolo bot e del suo filo di conversazione**: se un bot impara qualcosa, lo sa **lui**; gli altri no, a meno che non glielo diciate. Hermes ha **una** memoria, che vale in ogni contesto e sopravvive ai riavvii. La prova che lì è locale: quando esporti un team come pacchetto, il file porta i bot, le istruzioni, le skill e le routines — ma **non** credenziali, conversazioni, permessi, **memoria** né accesso al

computer.

2. **Non impara.** Non c'è un posto dove mettere una procedura scoperta una volta. Hermes scrive **skill**.
3. **È un'app da scrivania.** Non sta su un server come servizio con i suoi bot. Hermes sì.

Cosa ci fai, in concreto

Tre esempi, che sono i casi per cui è stato pensato.

Il riassunto della posta, ogni mattina alle nove. Dici al bot, in chat: *«ogni giorno feriale alle 9, mandami un riassunto della posta»*. Lui imposta l'orario da sé e da quel momento lavora anche a computer spento. La mattina trovi il riassunto. Se un giorno non ti serve, dalla carta nella chat lo metti in pausa.

La bozza di risposta, quando arriva la mail giusta. *«Quando arriva una mail da un certo cliente, prepara una bozza di risposta.»* Il bot collega l'evento e da quel momento la bozza è pronta prima che tu apra la posta. **Tu la leggi e la mandi**: il bot non spedisce niente da solo.

La squadra con un capo. Metti un bot come capo e sotto di lui tre specialisti — posta, spese, correzioni di codice. Il capo riceve la richiesta e la smista: *«questa è roba di spese»*. Gli specialisti si scambiano il contesto **fra loro**, in una chat di gruppo che puoi leggere. Tu intervieni solo quando uno chiede un permesso.

E il controllo, quando serve. Un bot sta lavorando su un sito: apri il suo desktop e lo **guardi muoversi** in tempo reale — dove clicca, cosa scrive, dove si blocca. Non è un riassunto di quello che ha fatto: è quello che sta facendo.

Lo stato, verificato oggi

I numeri che girano nei video **invecchiano in fretta**, e questo è un esempio perfetto: un video recente parlava di «oltre mille stelle su GitHub» e di «solo per Mac». **Nessuna delle due cose è più vera.**

	diceva il video	**al 28 ottobre 2026**
Stelle su GitHub	«oltre 1.000»	**5.769** (dalla fonte ufficiale: `milind-soni/OpenMausBot`)
Piattaforme	«solo macOS»	**macOS (due versioni: Apple silicon e Intel), Windows, Ubuntu 24.04**
Attività programmate	«segnaposto, non funzionano»	**funzionanti**: orari, webhook, eventi delle app
Programmi supportati	Claude e Codex	**Claude, Codex e Grok**
Versione	—	**v0.1.130** (27 ottobre), ultimo aggiornamento **28 ottobre**

	diceva il video	**al 28 ottobre 2026**
Fork · questioni aperte	—	972 fork · 487 questioni aperte

Perché la cifra è datata. Il progetto **si muove in fretta**: in due giorni ha guadagnato 144 stelle e ha pubblicato tre versioni. Una pagina di terze parti consultata il 26 ottobre ne dichiarava 5.625, e il 28 erano già 5.769. I numeri si prendono **dalla fonte ufficiale** (l'API del repository) e si **datano** quando si scrivono: una cifra senza data è una cifra che sta già invecchiando.

Le cose da sapere prima di provarlo: lo stato dichiarato dagli autori è *«presto, ma vero»* — il giro completo funziona (messaggio → agente → risposta → strumenti → approvazioni → uso del computer), ma **Ubuntu è in prova**, la connettività da telefono è ancora in costruzione, e i webhook usano un ricevitore locale invece di un servizio sempre acceso. La versione per Windows esiste, ma le capacità avanzate — controllo del computer, dettatura — sono documentate per macOS e Ubuntu, non per Windows.

> ***La lezione, che vale oltre questo caso**: i video invecchiano in settimane. Un progetto giovane cambia piattaforme, funzioni e numeri a un ritmo che nessuna trascrizione può seguire. **Prima di decidere, si guarda il repository**, non il video che ne parlava.*

18 · Pi: l'agente che ha deciso di non avere quasi niente

E qui il confronto con Hermes è ancora più netto, perché **Pi è l'esatto opposto**: è un harness agentico **minimale**, e la sua unica ambizione è fare il minimo indispensabile.

	Hermes	**Pi**
Che cos'è	agente completo, con harness proprio	**harness minimale**: il giro e quattro strumenti
Strumenti	decine: browser, voce, subagenti, lavori programmati	**quattro**: leggi, esegui, modifica, scrivi
Istruzioni di sistema	ampie (personalità, regole, procedure)	**poche righe**
Memoria	persistente fra le sessioni	**nessuna**: solo la sessione in corso
Skill	**1.428 file**	supporta i file di istruzioni, ma non ne porta
Permessi	delegati al modello	**nessuno: parte senza chiedere**
Connettori MCP	55	**non li supporta**

	Hermes	Pi
Subagenti	sì	**no** (se serve, lancia se stesso da riga di comando)
Lavori in sottofondo	sì	**no**: i comandi sono sincroni
Interfaccia	desktop, riga di comando, gateway	**terminale** (con interfacce grafiche della comunità)

Cosa fa, esattamente

Pi è un **assistente di programmazione da terminale**. È scritto da **Mario Zechner**, è libero (licenza MIT), e nasce da una frustrazione dichiarata senza giri di parole: gli strumenti che usava erano diventati *«un'astronave con l'80% di funzioni che non mi servono»*, e le loro istruzioni di sistema cambiavano a ogni versione, rompendo i flussi di lavoro.

La sua filosofia sta in una riga: **«se non mi serve, non lo costruisco»**. E la lista di quello che **non** ha è la cosa più interessante del progetto, perché ogni assenza è una **decisione**, non una mancanza.

Quello che non c'è, e perché

- **Niente elenchi di cose da fare.** L'agente non tiene una lista di attività: se serve, se la scrive in un file.
- **Niente modalità «pianifica prima».** Non c'è un passo separato in cui l'agente progetta e tu approvi: si lavora.
- **Niente connettori MCP.** Un intero standard di integrazione, escluso. Se serve uno strumento, si usa la riga di comando.
- **Niente comandi in sottofondo.** I comandi girano **sincroni**: non puoi avviare un server e continuare a parlarci. La motivazione dell'autore è che gestire processi in sottofondo porta complicazioni — seguire i processi, raccoglierne l'output, pulirli all'uscita — e che gli strumenti che lo fanno hanno **scarsa visibilità** su cosa sta girando.
- **Niente subagenti.** Non esiste uno strumento per generare un agente figlio. L'autore è esplicito: quando un altro agente ne genera uno, **tu non vedi niente** di quello che fa — «una scatola nera dentro una scatola nera» — e se sbaglia, capire perché è doloroso. Se ti serve, chiedi a Pi di **lanciare se stesso** da riga di comando, magari dentro una sessione di terminale che puoi guardare.
- **Niente permessi.** Pi parte **senza chiedere conferma**: gli strumenti girano con i privilegi dell'utente che l'ha avviato. È una scelta dichiarata, ed è la ragione per cui — se lo si usa da interfaccia grafica — va tenuto **solo su indirizzo locale**: quella è la rete di sicurezza, e vale la pena dirla invece di lasciarla intuire.

Quattro strumenti e poche righe di istruzioni

Tutto quello che Pi sa fare sta in **quattro strumenti**: leggere un file, eseguire un comando, modificare un file, scriverne uno nuovo. Le istruzioni di sistema sono poche righe: *«sei un*

assistente esperto; hai questi quattro strumenti; usa la riga di comando per le operazioni sui file; leggi prima di modificare; sii conciso». Fine.

Il vantaggio dichiarato è la **sobrietà**: poche istruzioni significano **meno gettoni spesi** e un comportamento più prevedibile. Ed è la stessa ragione per cui l'autore rivendica di poter **controllare davvero il contesto** — cioè cosa finisce sotto gli occhi del modello — cosa che gli altri strumenti rendono difficile iniettando roba che non si vede.

Cosa ci fai, in concreto

Quattro esempi, tutti di lavoro vero.

Rinominare duecento file. *«In questa cartella rinomina tutti i file da* `foto-1.jpg` *a* `vacanza-2026-01.jpg` *in ordine di data.»* Un comando, il controllo che siano tutti lì, fatto. Con quattro strumenti basta: leggere l'elenco, eseguire, verificare.

Sostituire una funzione in un file. *«In questo file la funzione* `calcola` *usa ancora il vecchio formato: aggiornala e controlla chi la chiama.»* Legge, modifica, cerca i richiami, ti dice cosa ha toccato. Nessuna cerimonia.

Cercare dove si usa una cosa. *«In tutti i file di questo progetto, dove viene usata questa variabile?»* Un comando di ricerca, il risultato, e la decisione è tua.

Spiegare un errore. *«Esegui il test e dimmi perché fallisce.»* Esegue, legge l'errore, apre i file che servono, ti spiega. Se la correzione è ovvia, la fa; se no, te la propone.

E cosa *non* gli chiedi. *«Ricordati che il cliente preferisce le mail brevi»*: non se lo ricorda, non ha memoria fra le sessioni. *«Metti in piedi un servizio che sorvegli una cartella e mi avvisi»*: non lo fa, i comandi sono sincroni e non ci sono lavori programmati. *«Chiedimi il permesso prima di cancellare»*: non chiede, parte e basta. Per queste tre cose serve un altro agente — ed è esattamente il confine con Hermes.

Come si aumenta l'harness (senza installare un altro programma)

Questa è la parte che sorprende chi arriva da strumenti chiusi. In Pi **non esiste un mercato di estensioni** da cercare: **si chiede a Pi di costruirsi la funzione che serve**. La frase sul sito è letterale — *«Pi non è un prodotto sigillato: se ti serve un comando, uno strumento, un fornitore, un flusso di lavoro o una modifica all'interfaccia, chiedi a Pi di costruirla. Si personalizzerà al volo»*.

Il giro è questo: chiedi la funzione in linguaggio naturale, Pi **scrive il codice dell'estensione**, tu la usi. Se poi vuoi condividerla con altri, diventa un **pacchetto Pi** installabile da npm o da git.

E i modi per aumentarlo sono cinque:

come	cosa aggiunge
Estensioni	comandi, strumenti, fornitori, flussi, modifiche all'interfaccia
Skill	pacchetti di istruzioni e strumenti, **caricati solo quando servono**

come	cosa aggiunge
File `AGENTS.md`	istruzioni di progetto, lette all'avvio dalla tua cartella, da quelle superiori e da `~/.pi/agent/`
File `SYSTEM.md`	sostituisce o integra le istruzioni di sistema, **per singolo progetto**
Modelli e fornitori	15 e più fornitori (Anthropic, OpenAI, Google, xAI, Groq, OpenRouter, Ollama…) e i tuoi, via un file o un'estensione

E poi c'è la **compattazione**, cioè il riassunto dei messaggi vecchi quando il contesto si riempie: in altri strumenti è una scatola nera, qui è **personalizzabile con un'estensione** — puoi far riassumere per argomento, o con un modello diverso, o in modo che capisca il codice.

In una riga: l'harness di Pi non è piccolo perché non si può allargare. È piccolo perché **parte da zero e cresce solo dove serve** — e la crescita la scrive l'agente stesso.

Un esempio concreto di estensione

Il caso più semplice che si fa davvero: **il salvataggio su GitHub in un comando solo**. Invece di digitare a mano la sequenza di comandi per preparare, scrivere il messaggio e pubblicare, chiedi a Pi:

> *«Fammi un comando che prende i file modificati, mi chiede il messaggio di commit e pubblica.»*

Pi scrive l'estensione, la installa, e da quel momento hai un comando tuo — con il suo pezzo di interfaccia se serve. Se ti va, la impacchetti e la condividi; se non ti serve più, la togli.

Altri esempi dello stesso tipo: un comando che apre il browser sulla pagina che stai modificando, un'estensione che aggiunge il tuo fornitore di modelli non previsto, una che cambia il modo in cui vengono riassunti i messaggi lunghi.

C'è un'interfaccia grafica? Sì, ma non è ufficiale

Qui serve precisione, perché la confusione è facile.

Pi, di suo, è da terminale. Il pacchetto ufficiale — `@earendil-works/pi-coding-agent`, degli autori di Pi — è **solo a interfaccia testuale**: non ha un'opzione per mettersi in ascolto su una porta o per servire una pagina web. Se cerchi «Pi» con un'interfaccia grafica, **non la trovi nel pacchetto ufficiale**: la trovi nella comunità.

L'interfaccia grafica esiste grazie alla comunità: un pacchetto `pi-web-ui` — di un autore diverso dagli autori di Pi, licenza libera, in sviluppo attivo — che avvia una pagina nel browser con:

- chat con risposta che scorre in diretta;
- **un terminale vero** dentro la pagina;
- pannello per Git, ricerca, **subagenti**, plugin, temi;
- funziona anche come **app installabile** dal browser (quelle che si aprono come programmi).

Due cose da sapere prima di usarla:

1. **Riusa la configurazione di Pi.** Stesse credenziali, stessi modelli, stessa cronologia: non devi riconfigurare niente, e quello che fai nel browser lo ritrovi nel terminale.
2. **Pi non ha un sistema di permessi.** Gli strumenti girano con i privilegi dell'utente che l'ha avviato: se ci metti davanti un browser, stai dando a quel browser **una riga di comando con i tuoi diritti**. È accettabile **solo perché resta in ascolto sul tuo computer** — ed è esattamente per questo che va tenuto su indirizzo locale e non esposto in rete.

E c'è una terza via: Pi ha **quattro modalità** — interattiva, a stampa, **a chiamate remote** e **come libreria**. La modalità a chiamate remote è il motivo per cui esistono interfacce di terzi: si può costruire la propria. È così che è nata quella della comunità, ed è così che **OpenClaw** — un altro strumento che si vede in giro — usa Pi dentro di sé.

Quando ha senso, e quando no

Ha senso se sai cosa stai facendo: se ti serve un assistente che legga, modifichi ed esegua, senza cerimonie, e vuoi vedere esattamente cosa gli passi. Costa poco in gettoni e non ti obbliga a imparare un sistema.

Non ha senso se ti serve un agente che **ricordi**, che **impari procedure**, che **giri su un server**, che **chieda permesso** prima di toccare qualcosa o che **coordini altri agenti**. Per questo esiste Hermes: sono due risposte opposte alla stessa domanda.

E il paradosso utile: Pi dimostra che **un harness grande non è obbligatorio**. Un agente che sa fare quattro cose può essere più utile di uno che ne sa fare cento — se le quattro sono quelle giuste e le cento ti confondono.

Come si combinano: una filiera completa

Mettiamo insieme i pezzi, in un caso reale: **un tutorial che mostra come si usa un'applicazione**.

1. **Si decide cosa mostrare.** Una descrizione scritta di quello che il video deve far vedere. È il testo che guiderà tutto il resto.
2. **L'automatismo entra nel servizio** (capitolo 12): finestra propria, accesso fatto dalla persona, sessione salvata.
3. **Si legge l'interfaccia** con Playwright (capitolo 3): quali elementi ci sono, dove sono, cosa dicono.
4. **Si decide, un passo alla volta, cosa mostrare** — e qui sta l'accorgimento che fa la differenza: non si pianifica tutto in anticipo, perché dopo il primo clic la schermata cambia. Si procede **una tappa alla volta**, rileggendo la pagina dove si è arrivati.
5. **Si registra con ffmpeg** (capitolo 1), non con lo screencast: così si vede il puntatore vero.
6. **Si scrivono le didascalie e si genera la voce** (capitolo 11) per ogni tappa.
7. **Si monta tutto** con ffmpeg: le dissolvenze fra le scene, la musica di sfondo, l'abbassamento automatico della musica quando la voce parla.

8. **Si esporta** nel formato di destinazione — 1920×1080 per una piattaforma didattica, verticale per i social.

Il punto importante è **il numero 4**. La differenza fra un video che mostra davvero come si usa un programma e un video che si limita a sorvolarlo sta tutta lì: nel decidere **una mossa alla volta**, guardando dove si è arrivati, invece di scrivere un copione e sperare che regga.

Le due fasi, e perché servono

Un ultimo accorgimento, che sembra un dettaglio tecnico ed è invece sostanziale: **non si registra mentre si pensa**.

Se l'automatismo registra mentre decide la mossa successiva, nel video finiscono i secondi in cui sta "pensando" — e diventano minuti di pagina ferma. Si lavora quindi in **due fasi**:

1. **prima si esplora senza registrare**: si scoprono le tappe e si preparano le voci;
2. **poi si registra** il percorso già deciso, dove l'unica attesa è quella del parlato.

Misurato su un caso reale: lo stesso percorso passa da oltre tre minuti a venti secondi. La differenza non è di stile, è di usabilità: un video di tre minuti per mostrare tre cose non lo guarda nessuno.

In una pagina: cosa serve, quando

Serve a	Strumento	Gratuito?	Licenza
Tutto il montaggio, la conversione, la registrazione	**ffmpeg**	**sì**	libera (LGPL/GPL) — non esiste a pagamento
Registrare lo schermo a mano	**Recordly** (recordly.dev)	**sì**	aperta (AGPL-3.0)
Far muovere il browser da solo, e registrarlo	**Playwright** + ffmpeg	**sì**	aperta (Apache-2.0)
Animare schede e testi	**GSAP**	**sì, anche per lavoro**	**proprietaria**, non aperta
Oggetti 3D	**Meshy**	100 crediti/mese, 10 scarichi	gratis: CC BY 4.0 (citare) · a pagamento: senza attribuzione
Immagini e video generati in locale	**ComfyUI**	**sì**	aperta (GPL-3.0) — ma serve una GPU vera
Immagini e video generati nel cloud	**Comfy Cloud**	5 prove, poi da ~20 $/mese	commerciale

Serve a	Strumento	Gratuito?	Licenza
Video fatti interamente di codice	**Lemo-Opuscar**	**sì**	aperta (MIT; demo CC BY 4.0, citare)
Registrare lo schermo dal browser, senza installare	**AnyMP4 online**	**sì, senza filigrana**	commerciale (max 24 fps)
Registrare lo schermo dal browser (con filigrana)	**AnyRec online**	sì	commerciale
Conversione e montaggio con i pulsanti	**AnyMP4 / AnyRec desktop**	prova con limiti	a pagamento (~12-15 $/mese)
Immagini generate via indirizzo web	**Pollinations**	**sì, senza chiave**	codice aperto (MIT), servizio a crediti
Testo generato	**Pollinations**	indirizzo semplice sì, API no	come sopra
Entrare in un servizio senza dare la password	**Accesso delegato**	tecnica, non prodotto	—

Le regole che valgono sempre

1. **Distingui "gratuito" da "aperto".** Un programma può essere gratis oggi e a pagamento domani; uno aperto, con una licenza libera, non può tornare indietro. Quando puoi, scegli l'aperto.
2. **Controlla il catalogo, non l'etichetta.** "Gratis" senza specificare *cosa* è una parola vuota: le immagini possono essere gratuite e il testo no, la versione locale gratuita e quella nel cloud a pagamento.
3. **Verifica la licenza prima di usare il risultato in un lavoro.** Vale per i modelli 3D (Meshy: con il piano gratuito devi **citare la fonte**), per le librerie di animazione (GSAP è gratis ma **non è aperto**), per le immagini generate. È il dettaglio che si scopre troppo tardi, quando il lavoro è finito.
4. **La base è ffmpeg.** Se impari quello, gli altri strumenti diventano interfacce comode attorno a qualcosa che sai già fare — e puoi cambiarli senza ricominciare da capo.
5. **Mai un vicolo cieco.** Vale per il software e vale per chi lo scrive: ogni errore deve indicare una strada, non solo un muro.

6. **La credenziale resta a chi appartiene.** L'automatismo riceve un permesso, non una password. E il permesso si può togliere.
7. **Verifica l'aggiornamento, e cerca quello che manca.** Le informazioni su questi strumenti **invecchiano in settimane**: un video, un articolo o un appunto di tre mesi fa può descrivere un programma che nel frattempo ha cambiato piattaforme, prezzi, funzioni e perfino nome. Prima di decidere — e prima di scrivere — si guarda la **fonte ufficiale** (il repository, il catalogo dei modelli, la pagina del progetto) e si controlla: **cosa è cambiato**, e **cosa c'è di nuovo che prima non c'era**.
8. **La fonte ufficiale batte il racconto.** Vale per i numeri (le stelle di un progetto, il prezzo di un modello, le piattaforme supportate) e vale per le promesse: quando la pagina di presentazione e la documentazione tecnica non dicono la stessa cosa, **ha ragione la documentazione**. Le due cose non coincidono quasi mai per caso.